

This set provides a varied spectrum of behaviors with differing curvature, boundedness, and gradient characteristics. Our model aims to select the most appropriate function at the penultimate layer by routing dynamically during training.

3.1 The Gumbel-Softmax Trick

Optimization involving discrete choices poses a major challenge to gradient-based learning. The Gumbel-Softmax trick (Jang et al., 2017) provides a continuous relaxation to sampling from categorical distributions, allowing for low-variance gradient estimation via reparameterization.

Given class probabilities $\pi_1, \dots, \pi_k$, the Gumbel-Max trick samples:

$$z = \text{one_hot} \left(\arg \max_i [\log \pi_i + g_i] \right), \quad g_i \sim \text{Gumbel}(0, 1),$$

where $g_i = -\log(-\log u_i)$ and $u_i \sim \text{Uniform}(0, 1)$.

Replacing $\arg \max$ with a softmax yields the Gumbel-Softmax distribution:

$$y_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum_j \exp((\log \pi_j + g_j)/\tau)},$$

where τ controls the "sharpness" of selection. As $\tau \rightarrow 0$, this approaches a true categorical distribution; higher τ yields smoother mixtures. The Gumbel-Softmax enables gradient flow through discrete selections, and has been used in areas ranging from generative models (Kusner & Hernández-Lobato, 2016) to channel pruning (Strypsteen & Bertrand, 2021). We employ it to the appropriate activation function suitable for the model without any overhead on hyperparameter search.

Gradient Estimation. The key advantage of the Gumbel-Softmax is that it allows gradients with respect to the logits $\log \pi_i$ to be computed via backpropagation. However, as we show in Section 3.3, naive use leads to activation selection biased toward unbounded functions like ReLU. This motivates the normalization strategy introduced in Section 3.3.

3.2 Activation Selection via Soft Routing

We now define the proposed model. At each layer i , given input X_{i-1} , we first compute the affine transformation $h_i = W_i X_{i-1} + b_i \in \mathbb{R}^{d_{\text{out}}}$. Rather than applying a fixed non-linearity, we compute a convex combination over a set of candidate functions at the penultimate layer:

$$X_i = \sum_{j=1}^p p_i^{(j)} \cdot \sigma^{(j)}(h_i),$$

where $\mathbf{p}_i \sim \text{GumbelSoftmax}(\log \boldsymbol{\pi}_i, \tau)$. Each $\sigma^{(j)}$ is a pre-defined activation function, and the logits $\boldsymbol{\pi}_i \in \mathbb{R}^p$ are trainable parameters learned jointly with the network weights. As τ is annealed, the model transitions from exploring blends to committing to specific activations per layer.

Straight-Through Estimator. During training, we optionally employ a straight-through estimator (Bengio et al., 2013), passing discrete one-hot samples in the forward pass while propagating gradients through the softmax relaxation. This enables consistent behavior between training and inference while maintaining end-to-end differentiability.

3.3 Bias Correction via Gradient Normalization

Scale-Induced Selection Bias. In preliminary experiments, we observed that the model consistently favored ReLU and Leaky ReLU where the activations have large unbounded outputs regardless of data-specific structure. This arises because backpropagation scales the gradient by the activation value itself, biasing selection toward functions with large magnitudes rather than functions more appropriate for the data regime.

Weighting with Gradient Norms. To address the problems observed with the gradient, we introduce a gradient normalization mechanism to prevent this.

We compute the gradient norms of each activation function with respect to the input:

$$g_i = \|\nabla_x \phi_i(h)\|_2, \quad (1)$$

where $h = \bar{\mathbf{X}}\mathbf{W}^\top + \mathbf{b}$.

We convert the negative average gradient norms into pseudo-probabilities using a softmax function:

$$\tilde{p}_i = \frac{\exp(-\bar{g}_i/\lambda)}{\sum_{j=1}^K \exp(-\bar{g}_j/\lambda)}, \quad (2)$$

where $\bar{g}_i$ is the average gradient norm for activation function ϕ_i over the batch, and λ is a scaling factor to adjust the pulling factor.

The total loss is then a combination of the primary task loss (e.g., mean squared error for regression) and the Kullback-Leibler divergence between the model’s activation probabilities and the pseudo-label probabilities:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \alpha \mathcal{L}_{\text{KL}}, \quad (3)$$

where

$$\mathcal{L}_{\text{KL}} = \sum_{i=1}^N \sum_{k=1}^p \tilde{p}_i^{(k)} \log \left(\frac{\tilde{p}_i^{(k)}}{p_i^{(k)}} \right), \quad (4)$$

and α is a weighting coefficient. By including this regularization term, we were able to overcome the problems with the bias towards suboptimal activation functions in our experiments.

4 Experiments

In this section, we present an empirical evaluation of **Flex-Act**, our proposed dynamic activation selection framework. We compare it against standard fixed-activation networks and ablate key components of our model, including the use of regularization. Our experiments are designed to isolate the role of activation selection in shallow regression tasks, where an optimal solution is analytically known. This controlled setting allows us to precisely measure convergence, interpretability, and the effect of discrete selection dynamics.

4.1 Experimental Setup

We construct a synthetic regression task where the target label is generated via a non-linear activation applied to a single informative input variable, with additional distractor features. Specifically, we generate input vectors $x \in \mathbb{R}^4$, where one dimension x_1 is informative, and the remaining are i.i.d. Gaussian noise. The label is given by:

$$y = a(k \cdot x_1),$$

where $a(\cdot)$ denotes the ground truth activation function, and k is constant, which we arbitrarily set to 5 for the sake of simplicity throughout our experiments. We test five such cases for activation functions: **ReLU**, **Sigmoid**, **Tanh**, **LeakyReLU**, and **Identity**. Our goal is to evaluate whether models can recover this target transformation without any sweep and computational overhead.

We compare the following model variants:

- **Flex-Act (Ours):** A single-layer network with dynamic, layer-wise activation selection using Gumbel-Softmax sampling from five candidate functions. Details are provided in Section 3.
- **Fixed-Activation Networks:** Five networks, each using a fixed activation from the candidate set. This provides an upper bound when the activation matches $a(\cdot)$, and a lower bound when mismatched.

Each model is trained to minimize Mean Squared Error (MSE), and we report both final error and convergence behavior averaged over 5 different seeds. For **Flex-Act** models, we additionally track the evolution of the activation selection probabilities over time for additional insights.

4.1.1 Results and Analysis

Performance Across Ground Truths. Table 1 reports the mean and standard deviation of the MSE for each model across different ground truth functions. As expected, each fixed-activation model achieves zero error only when its activation matches the ground truth. In contrast, **Flex-Act** consistently converges to the optimal function, matching or outperforming all baselines in every setting without any additional computational overhead. This advantage helps circumvent the need to train different models for each activation functions and provides both interpretability and efficiency in training.

Table 1: Mean and standard deviation of MSE across various ground truth activations. The best result per column is highlighted in **bold**. The best result for each metric is shown in **bold**, and the second-best is marked with $\dagger$.

Model	ReLU	Sigmoid	Tanh	LeakyReLU	Identity
Flex-Act ($\alpha = 0.3$)	$0.0001 \pm 0.0002^\dagger$	$0.0011 \pm 0.0006^\dagger$	$0.0001 \pm 0.0002^\dagger$	$0.0001 \pm 0.0002^\dagger$	0.0000 ± 0.0000
Flex-Act ($\alpha = 0.0$)	$0.0001 \pm 0.0001^\dagger$	0.0059 ± 0.0073	0.0021 ± 0.0028	$0.0001 \pm 0.0002^\dagger$	0.0000 ± 0.0000
ReLU Model	0.0000 ± 0.0000	0.0059 ± 0.0072	0.4328 ± 0.4287	0.0004 ± 0.0007	4.1675 ± 6.7199
Sigmoid Model	2.1360 ± 3.9912	0.0000 ± 0.0000	0.4020 ± 0.4536	2.1365 ± 3.9910	6.3056 ± 6.5772
Tanh Model	2.2941 ± 3.9616	0.0141 ± 0.0119	0.0000 ± 0.0000	2.2909 ± 3.9623	4.2737 ± 4.7666
LeakyReLU Model	0.0004 ± 0.0007	0.0059 ± 0.0072	0.4286 ± 0.4227	0.0000 ± 0.0000	4.0787 ± 6.5640
Identity Model	0.5209 ± 0.4659	0.0078 ± 0.0061	0.0985 ± 0.0798	0.5105 ± 0.4566	0.0000 ± 0.0000

Importantly, the variant without regularization (**Flex-Act** with $\alpha = 0$) performs significantly worse when the ground truth is **Sigmoid** or **Tanh**. This confirms our earlier hypothesis that unbounded activations (e.g., ReLU) dominate gradient flow during training, leading to biased selection without proper correction.

Selection Dynamics. Figure 1 tracks the Gumbel-Softmax probability distribution over time. In each case, the model quickly converges toward the correct activation, sometimes switching transiently between close candidates (e.g., ReLU and LeakyReLU).

Effect of Regularization. Figure 2 shows the selection dynamics with and without gradient-based regularization. Without it ($\alpha = 0$), the model lingers on ReLU or LeakyReLU due to their unbounded gradients. With regularization ($\alpha = 0.3$), the model rapidly discovers and locks onto the true activation.

5 Discussion

Our experiments validate that **Flex-Act** is capable of discovering the optimal activation function in a purely data-driven manner. Even in simple synthetic tasks, this flexibility enables better fit and interpretability than fixed or parameterized alternatives. Importantly, we show that naive use of Gumbel-Softmax leads to biased selection, and that our gradient-norm-based regularizer effectively corrects this issue. The interpretability of **Flex-Act** is a byproduct of its design: discrete selection exposes layer-wise preferences, enabling post hoc analysis and debugging agnostic of the task at hand. While such properties are often overlooked in deep learning systems, we find them critical for understanding and correcting unexpected optimization behavior, and could open up further interesting research avenues into choices of activation functions in deep learning literature. The current proposal of **Flex-Act** is primarily designed to be only added at the penultimate layer. We aim to extend **Flex-Act** to deeper architectures with multi-layer activation function selection, where different layers may benefit from different non-linearities. However, the current heuristic approach would face issues with gradient alignment, exploding gradients, and unstable training if extended naively. Furthermore, our current regularizer is heuristic and derived from empirical intuition; future efforts could explore alternative formulations grounded in information theory or activation smoothness metrics.

6 Conclusion

We introduced **Flex-Act**, a framework for discrete activation selection that enables a layer of a neural network to dynamically select its activation function during training. By leveraging the Gumbel-Softmax